

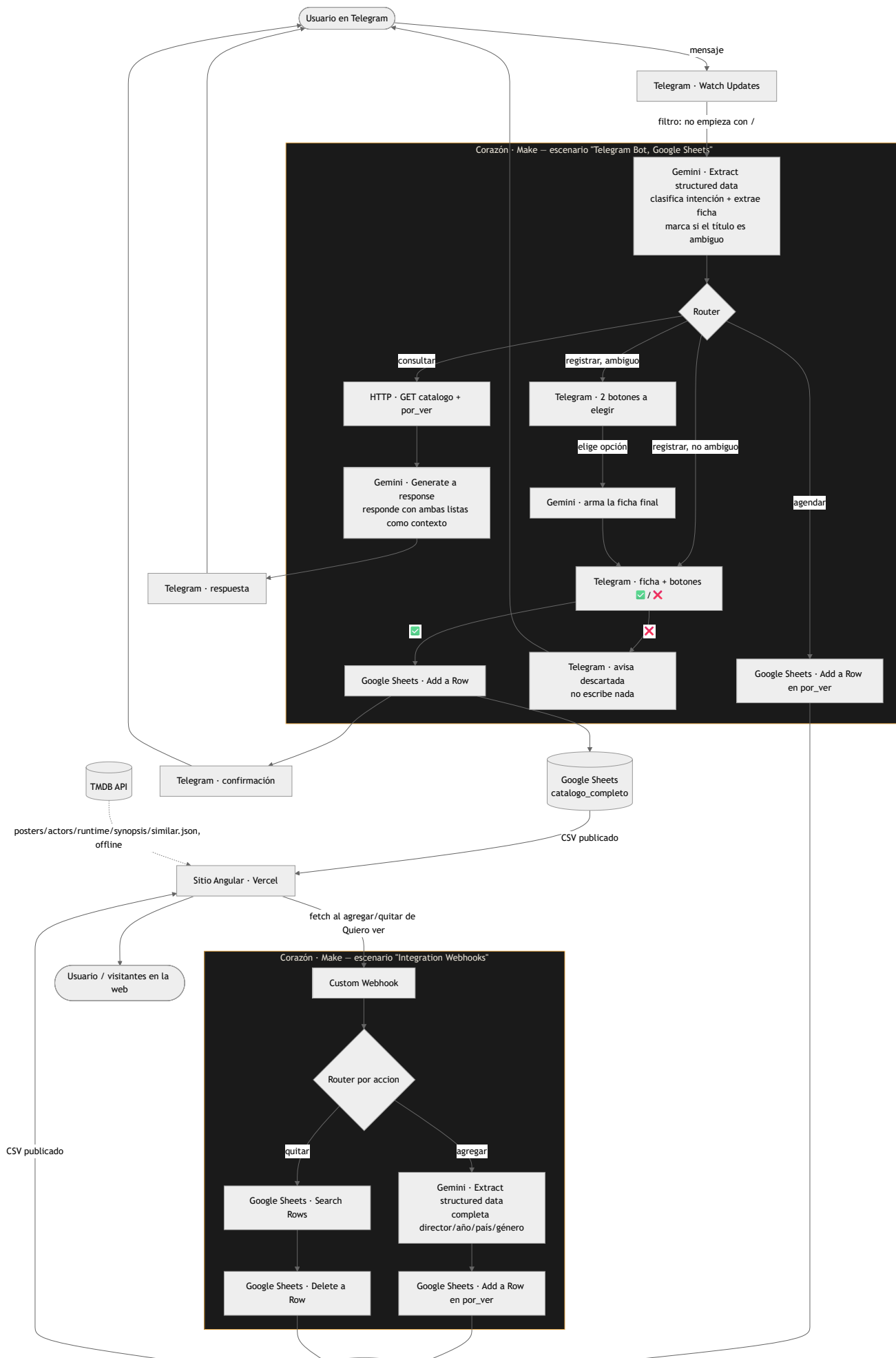
Mapa de arquitectura

Proyecto **Diario de proyección** — ecosistema autónomo de punta a punta para registrar y consultar un catálogo personal de películas.

Las tres capas

Capa	Qué es	Herramienta
Cerebro — los datos	Dos tablas: <code>catalogo_completo</code> (~1.290 filas, una por visionado) y <code>por_ver</code> (watchlist, ~60 filas). Fuente de verdad única.	Google Sheets
Corazón — la lógica	Dos escenarios de Make. "Telegram Bot, Google Sheets": escucha el chat, clasifica la intención, llama a la IA, y según el caso escribe/lee una fila o responde. "Integration Webhooks": recibe pedidos del sitio (agregar/quitar de <code>por_ver</code>) por un webhook propio.	Make + Google Gemini
Voz — entrada y salida	El chat para registrar, agendar y preguntar; la web para explorar el catálogo, ver estadísticas y manejar la lista "Quiero ver".	Telegram + sitio en Angular (Vercel)

Diagrama de flujo





Los caminos

1. Registrar una película (con HITL, con desambiguación)

"vi Whiplash"

- Watch Updates capta el mensaje
- filtro descarta comandos (/start...) · Router: solo mensajes de texto van a Gemini
- Gemini clasifica: intent = registrar; extrae titulo_corregido, director, anio_estreno, pais_origen, genero; marca ambiguo = true/false
- si ambiguo = true (ej. "vi cape fear"): Telegram manda 2 botones (la original de 1962 / el remake de 1991) → al elegir, un segundo llamado a Gemini arma la ficha completa sobre esa opción
- Router → ficha lista → Telegram: manda la ficha con botones [☒ Sí] [☐ No]

(el usuario toca ☒)

- Watch Updates capta el callback_query · Router → rama "Aprobado"
- Add a Row en catalogo_completo (fórmulas por-fila para id, numero, es_revisionado; fecha_vista = hoy; fuente = Bot Telegram; estado_enriquecimiento = Aprobada)
- Telegram: " ☒ Guardada: Whiplash"

(el usuario toca ☐)

- Router → rama "Rechazado": no escribe nada
- Telegram: avisa que se descartó, invita a reintentar si fue sin querer

2. Agendar ("quiero ver")

"quiero ver Perfect Blue"

- Gemini clasifica: intent = agendar; extrae titulo/director/anio/pais
- Router → rama "agendar" → Add a Row en por_ver (sin paso de HITL)

3. Consultar / recomendar

"¿qué vi de Cronenberg?" · "¿qué tengo pendiente por ver?" · "recomendame un thriller"

- Gemini clasifica: intent = consultar; consulta = pregunta reformulada
- Router → rama "consultar"
- HTTP GET al CSV publicado de catalogo_completo + HTTP GET a por_ver
- Gemini responde con ambas listas como contexto, distinguiendo vistas de pendientes (para recomendaciones: sugiere títulos que NO están en ninguna de las dos)
- Telegram: la respuesta

4. Agregar/quitar de "Quiero ver" desde el sitio

Click en "+ Agregar película" (sitio) → POST al Custom Webhook con el título

- Router por accion = "agregar" → Gemini completa director/año/país/género a partir del título libre (mismo patrón que el bot) → Add a Row en por_ver

Click en la X de una card / "Quitar de la lista" en el modal → POST con accion = "quitar"

- Search Rows (título + año) → Delete a Row en por_ver

El sitio

App en Angular 20 (carpeta `app/`, standalone + signals). Al cargar:

1. `fetch` al CSV publicado de `catalogo_completo` y de `por_ver` → datos en vivo (cae a una instantánea del catálogo, `app/public/data/catalog-snapshot.json`, si no hay conexión).
2. `fetch` a `posters.json` + `posters-manual.json` + `actors.json` + `runtime.json` + `synopsis.json` → póster, rating, género, reparto, duración y sinopsis por título (de TMDB, generado offline; los que TMDB no tiene o matchea mal se cargan a mano en `img/` + `posters-manual.json`). Más `similar.json`, `similar_ajustado.json` y `taste_profile.json` (recomendador y perfil de gusto, generados offline con Python). Todos se leen directo del repo de GitHub.
3. Renderiza: tiles de stats, gráficos SVG (sin librería), placas temáticas (incluida una de data cleaning, "Detrás de los datos"), índice completo en vista lista o grilla, sección "Quiero ver" paginada (con alta/baja propia) y un modal de detalle por película (sinopsis, reparto, parecidas).

Cada `git push` a `main` redeploja Vercel solo. Los JSON de datos se leen directo del repo, así que actualizarlos no necesita redeploy.

Componentes y dónde vive cada cosa

Componente	Ubicación	Notas
Bot	@ListadoPelisBot (Telegram)	creado con BotFather
Orquestación (bot)	Make · escenario "Telegram Bot, Google Sheets"	1 trigger, router de intención, ramas registrar/agendar/consultar

Componente	Ubicación	Notas
Orquestación (sitio)	Make · escenario "Integration Webhooks"	1 Custom Webhook, router por <code>accion</code> (agregar/quitar)
IA	Google Gemini (<code>gemini-3.1-flash-lite</code>)	varias llamadas: clasificar/extraer, desambiguar, responder, completar título libre del sitio
Base de datos	Google Sheet · pestañas <code>catalogo_completo</code> y <code>por_ver</code>	publicadas como CSV para lectura
Datos enriquecidos	<code>posters.json</code> , <code>actors.json</code> , <code>runtime.json</code> , <code>synopsis.json</code> , <code>similar.json</code> (TMDB) + <code>posters-manual.json</code> + <code>img/</code>	generados por scripts en <code>cerebro/</code> , ver manual-de-datos.md
Sitio	<code>app/</code> (Angular) en Vercel	<code>turimoviesdatabase.vercel.app</code>
Secretos	conexiones de Make (token de Telegram, key de Gemini) · <code>cerebro/tmdb.key</code> (local, ignorado)	nada de esto en el repo

Requisitos del proyecto integrador

Requisito	Cómo se cumple
Autónomo	El circuito Telegram → IA → Sheets → web corre solo, sin intervención.
Resiliente	<i>Pendiente:</i> falta Error Handler en Make (si Gemini devuelve 503, hoy se pierde la fila).
Con HITL	✅ "vi X" → el bot propone la ficha con botones  /  ; recién escribe (como <code>Aprobada</code>) al tocar  . La rama del  no guarda nada y avisa que se descartó.
Limpio	Una sola fuente de verdad (la hoja); filtro anti-comando; secretos fuera del repo.